



Compiladores

Aula 9: Análise Sintática Descendente

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

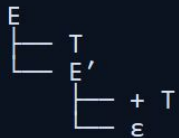
- Recursão à esquerda
- Fatoração à esquerda
- First e Follow
- Da gramática à AST

Recursão à esquerda

- As etapas do parser com recursão:
 - **Compreender:** Explicar a análise top-down e a descida recursiva como construção de uma derivação mais à esquerda.
 - **Transformar:** Eliminar recursão à esquerda e fatorar prefixos comuns sem perder a linguagem gerada.
 - **Predizer:** Calcular FIRST/FOLLOW e justificar escolhas do parser com base no lookahead.

Recursão à esquerda

- **Parsing top-down:** a árvore é expandida da raiz para as folhas, acompanhando o próximo token.



Derivação mais à esquerda

Em cada passo, o parser escolhe uma produção para o não-terminal mais à esquerda da forma sentencial.

Recursão à esquerda

- **Descida recursiva:**
 - um procedimento para cada não-terminal;
 - o scanner fornece tokens e o parser verifica a estrutura.

```

procedure A() :
  escolher A → X1 X2 ... Xn
  para cada Xi:
    se Xi é não-terminal:
      chamar Xi()
    senão:
      casar Xi com lookahead
  se falhar: sinalizar erro
  
```

Scanner

Entrega token atual, lexema e posição.

Parser

Decide expansões e consome tokens.

MINIC

parseProgram • parseStatement • parseExpression

Recursão à esquerda

- Se a chamada recursiva ocorre antes de consumir um token, a descida pode não terminar.

$E \rightarrow E + T \mid T$
 $\text{parseE}() \rightarrow \text{parseE}() \rightarrow \dots$

transformar

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \epsilon$

A linguagem é preservada; a forma da derivação muda para favorecer a predição.

Recursão à esquerda

- **Eliminação da recursão direta:** consiste em separar a alternativa-base da repetição recursiva.

$$A \rightarrow A \alpha \mid \beta$$

Intuição

β inicia a construção; A' representa zero ou mais ocorrências de α . O ε encerra a repetição.

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

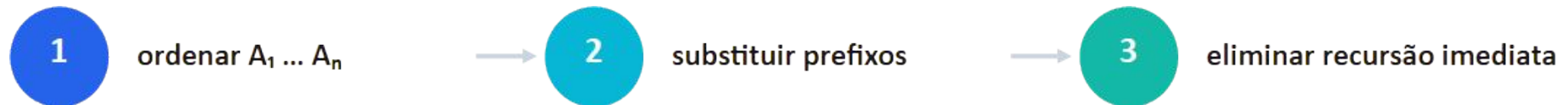
$$E \rightarrow T E' \quad E' \rightarrow + T E' \mid \varepsilon \quad T \rightarrow F T' \quad T' \rightarrow * F T' \mid \varepsilon$$

Recursão à esquerda

- Quando o ciclo passa por outros não-terminais:
a recursão pode aparecer em dois ou mais passos de derivação.

$$S \rightarrow A a$$

$$A \rightarrow S d \mid b$$

$$S \Rightarrow A a \Rightarrow S d a$$


Fatoração à esquerda

- Prefixos comuns exigem que o parser leia um pouco mais antes de escolher a produção.

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \quad \# \text{ regras}$$

$$A \rightarrow \alpha A' \quad \# \text{ fatoração}$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Com essa fatoração, o prefixo α é reconhecido uma única vez; a escolha é adiada para A' . Isso melhora a predição, mas não elimina automaticamente ambiguidades.

Fatoração à esquerda

- Fatorando comandos iniciados por if.
- O prefixo compartilhado fica explícito na gramática.

Comando $\rightarrow$ if (Expr) Comando else Comando
 | if (Expr) Comando
 | while (Expr) Comando
 | id = Expr ;

Comando $\rightarrow$ if (Expr) Comando Comando'
 | while (Expr) Comando
 | id = Expr ;

Comando' $\rightarrow$ else Comando | ϵ

O “else” associado ao “if” mais próximo continua exigindo convenção ou gramática apropriada.

First e Follow

- $\text{FIRST}(\alpha)$ reúne terminais que podem aparecer como primeiro símbolo de uma cadeia derivada de α .

- Regras-base:

Terminal X : $\text{FIRST}(X) = \{X\}$

Produção $X \rightarrow \varepsilon$: incluir ε

Sequência: avançar enquanto o símbolo for anulável.

$F \rightarrow (E) \mid \text{id}$

$\text{FIRST}(F) = \{ (, \text{id} \}$

Se o lookahead pertence a $\text{FIRST}(\alpha)$, $A \rightarrow \alpha$ pode ser escolhida sem retrocesso.

FIRST transforma uma escolha gramatical em uma decisão baseada no próximo token.

First e Follow

- FOLLOW(A) reúne terminais que podem ocorrer imediatamente à direita de A.

$E \rightarrow T E'$

$F \rightarrow (E)$

FOLLOW(E) inclui)

FOLLOW(E') recebe FOLLOW(E)

Regras essenciais:

\$ em FOLLOW(S), para o símbolo inicial.

$FIRST(\beta) - \{\epsilon\}$ entra em FOLLOW(B) quando $A \rightarrow \alpha B \beta$.

Se $\beta \Rightarrow^* \epsilon$, propagar FOLLOW(A) para FOLLOW(B).

\$ representa o fim da entrada e também pode atuar como marcador de sincronização.

First e Follow

- **A escolha de ϵ :**

FOLLOW é decisivo quando uma produção anulável precisa ser usada.

$$E' \rightarrow + T E' \mid \epsilon$$

$$\text{lookahead} = + \rightarrow + T E'$$

$$\text{lookahead} \in \text{FOLLOW}(E') \rightarrow \epsilon$$

O próximo token informa se há mais operadores ou se a construção terminou. A mesma lógica sustenta tabelas LL(1).

FOLLOW também pode sugerir tokens de sincronização para recuperar erros.

Da gramática à AST

- A preparação da gramática organiza tanto as decisões quanto a construção da árvore abstrata.



Expressões

$E \rightarrow T E'$; laços ou nós binários preservam precedência e associatividade.

Comandos

if, while e atribuições escolhem alternativas por tokens iniciais distintos.

Chamadas

Identificador + "(" conduz à lista de argumentos e ao nó de chamada.

Da gramática à AST

- Uma atividade curta para ligar teoria, gramática, implementação e testes.

Registre:

token de lookahead • produção escolhida • resultado • mensagem de erro esperada

A • Gramática

Escolha cinco não-terminais. Elimine uma recursão à esquerda e faça uma fatoração.

B • Conjuntos

Calcule FIRST e FOLLOW; justifique ϵ , terminais e o marcador \$.

C • Testes

Use declaração, atribuição, $a + b * c$, chamada, bloco e comando inválido.

Da gramática à AST

- Conflitos revelam que o lookahead disponível não distingue as alternativas.

$$\text{FIRST}(\alpha_1) \cap \text{FIRST}(\alpha_2) \neq \emptyset$$

→ escolha ambígua

→ fatorar ou reestruturar

Checklist:

- Precedência separada?
- Recursão eliminada?
- Prefixos fatorados?
- Chamadas e indexação distintas?
- FIRST/FOLLOW calculados?